Castles in the Air

Extended chapter commentary

CHAPTER 5
Engines and enigmas

© John Longley 2023

This version 2 December 2023

CHAPTER 5 COMMENTARY

Engines and enigmas

In the last chapter, we looked at one of the most basic ingredients of mathematics — the concept of a *mathematical statement* — and saw how this concept could itself be made mathematically precise (at least within certain domains), opening up the possibility of giving decision algorithms for parts of mathematics. In this chapter, our purpose is to apply a similar treatment to another basic ingredient — the concept of *computability.* As we saw at the end of the last chapter, a precise mathematical definition of this concept will be essential if we're to stand any chance of proving that some classes of mathematical statements are *undecidable*.

In our story, we explored one possible way of pinning down the concept of computability, using the idea of tube machines. We then saw that there were problems not solvable by tube machines (such as the halting problem) — from which it follows that there can be no tube machine that functions as a Decider for Whole number glyph statements.

So far so good. But why should results about tube machines hold any kind of fundamental importance? After all, tube machines are just one highly specific type of 'machine', built up from a very particular set of basic components that we've described, and there's nothing very special about them — you could easily dream up as many other kinds of 'machines' for step-by-step, mechanical computation as you wanted. But what we'd really like to show is that no purely mechanical computation procedure of *any* kind — no *algorithm* in the sense we introduced in Chapter 2 — could decide Whole number glyph statements in the way that Emily (and Hilbert) wanted. To reach this more sweeping conclusion, we would need to convince ourselves of the following:

Anything that's computable by any mechanical algorithm whatsoever is also computable by a tube machine.

We touched on this idea briefly towards the end of our story (recall that *engines* represent algorithms). If we can satisfy ourselves of this rather bold claim, then results about what can and can't be done by tube machines really can be taken to say something about what can and can't be done by algorithms in general.

The above claim is in essence what is known as the *Church-Turing thesis*, after the American logician Alonzo Church who first proposed it in 1935, and the British mathematician Alan Turing who arrived independently at an equivalent idea in 1936.[1] (Church and Turing didn't actually phrase their ideas in terms of our tube machines, but in terms of other mathematical gadgets of the same computational power — we'll come back to this below.) Because of its central importance, we'll begin this section with a detailed discussion of the 'thesis' — what we mean by it, and why one might believe it — before discussing some of the other ideas introduced in the story, such as Turing's idea of a Universal Machine. At the end of the chapter, we'll consider some variations on the Church-Turing thesis — which in turn will lead us into some intriguing and perplexing questions concerning computability and the physical world we live in.

The 'clerk-in-a-box' model

Why should we believe the Church-Turing thesis? Is it even totally clear what we mean by it? Let's tackle this latter question first.

There is a rather slippery problem here. As things stand, the thesis (in the form stated above) isn't something that's susceptible to a straightforward mathematical *proof*, because whilst there's no problem in making the definition of tube machines mathematically precise, we haven't given a precise definition of 'mechanical algorithm': our explanation of this concept in Chapter 2 was rather intuitive and informal. Indeed, the whole point of the thesis is to claim that a certain *intuitive* concept is well captured by a certain *precisely defined* one; and such a claim, by its very nature, doesn't lend itself to a rigorous proof *before* the intuitive concept has been more precisely defined.[2] The best we can do, then, is to start by trying to get the (perhaps initially vague) concept of 'computability by a mechanical algorithm' as clear in our minds as we can, and then to ask what reasons there might be for believing that this might coincide with computability by a tube machine.

These days, the use of the word 'mechanical' might make us instantly think of digital computers, but that wasn't what Church and Turing themselves originally had in mind. Rather, they were thinking of the kinds of computation that a *human being* could in principle carry out by following purely 'mechanical' instructions, and given unlimited time and resources. In other words, they were thinking about what could in principle be accomplished by a 'clerk' (as such a figure is sometimes rather quaintly called) of infinite diligence, patience and carefulness — some totally reliable chap (I'll take him to be male on this occasion) who never makes mistakes and never complains about any amount of hard, tedious work — sitting in a room with inexhaustible supplies of paper and ink, and mysteriously endowed with an

unlimited lifespan. We give the clerk two things: an *input* (let's say a Whole number, though it could equally well be a finite sequence of symbols drawn from some finite alphabet), and a finite set of *computing instructions* in the form of definite, precise rules that tell him what to do with this input. These instructions must leave no room for choices, subjective judgements or the exercise of creative imagination, but must intuitively be of an entirely routine or 'mechanical' character. Here's one example of a possible set of computing instructions:

> Given an *input* Whole number N, construct a sequence of numbers
> $a_0, a_1, \ldots a_{N+2}$ in the following way:
> Set $a_0 = 0$ and $a_1 = 1$.
> Then, for each Whole number i from 2 up to $N + 2$ in turn,
> set $a_i = a_{i-2} + a_{i-1}$.
> Once all of $a_0, a_1, \ldots a_{N+2}$ have been constructed,
> *output* the value of a_{N+2} as the final answer.

You might recognize these as instructions for computing what are known as *Fibonacci numbers*. We've assumed here that our clerk is familiar with some method for adding two Whole numbers; but if we were worried that he might not be, we could provide a description of some mechanical method for that too (such as the school method of addition). In any case, we can give the clerk his instructions, along with an input number N — let's say 10 — and leave him to get on with it. After a certain period of time, he will announce that his task is completed, and will deliver a slip of paper giving the *output* of the calculation — in this case, the number 144. The question we're asking, then, is about what can be computed in this way by *humanly executable routine procedures* of this kind.

There's one further point to clarify. It's not actually the concept of a 'routine procedure' itself that we're trying to pin down — only the extent of what's *computable* by such procedures. To understand this, we can imagine that once we've set up the clerk with his computing instructions, we enclose him and his entire room in a large black box. We are then able to feed him with various input numbers through a small hole in the wall, and he can deliver the output numbers through another hole. Someone who can't see inside the black box won't have any idea what instructions the clerk is following — all they'll be able to see is that the box as a whole functions as a device that transforms certain inputs into certain outputs. Such an observer might start to draw up a table to describe the box's behaviour, like this:

Input	0	1	2	3	4	5	6	….

Output	1	2	3	5	8	13	21	….

Of course, since there are infinitely many possible inputs, this table would have to be continued indefinitely to give a *complete* description of the box's behaviour. And although our observer could never actually construct it within a finite period of time, we can nevertheless imagine such an infinite table representing the 'operation' — or in standard mathematical terminology, the *function* from Whole numbers to Whole numbers — that the box as a whole computes.

In just the same way, we can also imagine enclosing a tube machine in a black box with an input and output hole, so that only the operation that it computes is visible to an outside observer. (To be more explicit, let's suppose that the input number gets inserted into the tube machine's A register, the other registers being initially set to 0 and available as 'workspace' for the machine. The disc is then run through the machine, after which the final value of the A register is extracted and posted through the output hole.) Again, we can imagine an infinite table that describes the mathematical function computed by this tube machine. We can now articulate our version of the Church-Turing thesis a bit more precisely as follows:[3]

Any function on Whole numbers
that can be computed by a clerk-in-a-box (with suitable instructions)
can also be computed by a Tube-Machine-in-a-box.

Notice that we're not saying here that the tube machine has to compute the operation in anything like the same *way* as the clerk does — only that the behaviour of the two boxes as seen from the outside is the same[4]. And even then, we don't care whether the tube machine is a million times faster than the clerk, or a trillion times slower — all we're interested in is whether the two boxes compute *the same operation*, in the sense that when fed with the same input number they will both (eventually) return the same output number. It's in this sense that we're claiming that tube machines have the same 'computing power' as clerks.

Incidentally, it's not hard to convince ourselves that clerks have *at least* the power of tube machines, because a clerk could easily be instructed to simulate the behaviour of any tube machine we gave him. The much more interesting question is whether tube machines can do everything clerks can. If we can convince ourselves of this, we'll be able to conclude that the decision problem for Whole number glyph statements (for instance) is indeed beyond the reach of the clerkly style of computation.

The case for the Church-Turing thesis

We've spent some time trying to clarify what the Church-Turing thesis is supposed to mean, but so far we haven't advanced any reasons for believing it. Why *should* we suppose that our humble tube machine kit can give rise, in principle, to the enormous range of computable functions of which our worthy clerk is capable?

At least three kinds of arguments in favour of the thesis have been suggested (in fact, all three were mentioned in Turing's original paper). The first is very simple: thousands of recognizably 'mechanical' procedures for calculating various things have been put forward, but no one has ever come up with one that *can't* be mimicked by a tube machine or something equivalent. So there is at least some kind of *cumulative* evidence for the thesis, and for many people, this is already pretty compelling. However, as we saw in Chapter 1, mathematicians as a breed tend to be notoriously unimpressed by such cumulative evidence. Perhaps there are other kinds of recognizably 'mechanical' procedure that are just very hard to think of for some reason, which is why no one's thought of them yet? In other words, couldn't there be some sort of systematic bias in the selection of examples we've so far encountered? Indeed, as we'll see in Chapter 7, there are other situations in logic where seemingly very strong cumulative evidence turns out to be misleading in just this kind of way. In the light of all this, it's therefore natural to ask whether we can find better arguments in favour of the thesis.

A second line of argument that's often been put forward hinges on the fact that there are many possible ways of *trying* to define the concept of a 'computable function' on Whole numbers (tube machines being just one), all of which can be shown to lead to exactly the same mathematical class of functions. To explain this point, it's worth taking a closer look at the history.

As we've mentioned already, Church himself didn't work with tube machines — rather, he was interested in two other kinds of mathematical gadgets, known as *recursion equations* and *lambda calculus.* Each of these could be used to define a class of functions on Whole Numbers, and Church was impressed by the fact that the two systems, despite their very different characters, gave rise to exactly the *same* operations. (The proof of this was mostly the work of Church's student Stephen Kleene.) Mathematicians tend to get excited when two apparently distinct lines of inquiry are found to converge on the same thing, and Church was led to suspect that there might be something important, something fundamental, about this particular class of functions.

Meanwhile, across the Atlantic, Turing was proposing yet another kind of computing gadget, now known as a *Turing machine*, and claiming that such machines were powerful enough to compute any 'mechanically computable' function whatever. Once again, the work of Kleene showed that Turing

machines gave rise to the exactly the same functions as Church's systems, further confirming the impression of the central importance of this class. Since then, many other logicians (such as Post and Markov) have suggested alternative approaches to the idea of computability, but in each case, their systems have turned out to have exactly the same computing power as those of Church and Turing. The tube machines I've used in the story are based on one such approach, that of *register machines*, proposed by Marvin Minsky in the 1960s.[5] In any case, the moral of all this is that the Church-Turing class of computable functions appears to be mathematically a very 'robust' one, in that the precise technical details of the mathematical gadgets you use don't seem to matter too much: any model of computation in roughly the right ballpark will lead you to the same class of computable operations.

This is all very well, and it reassures us that for the purpose of discussing the Church-Turing thesis, we can choose whichever of these gadgets we prefer. But does this convergence of different definitions really provide significant support for the thesis itself? Certainly, in the early days, some people seemed to feel that it did, but since then, the strength of this particular argument has been called into question.[6] After all, the situation of having many definitions leading to the same thing is quite common in mathematics — and whilst this may indeed show that the Church-Turing computable functions form a mathematically 'natural' class in some way, that doesn't by itself say anything about whether they have anything to do with the informal idea of 'mechanical computability' — unless at least some of these definitions *individually* can be convincingly argued to capture this idea. But can they?

This brings us to the third line of argument in favour of the Church-Turing thesis — the one which many find the most compelling. This argument was developed at some length by Turing itself, and came from reflecting carefully on the nature of what our stalwart clerk would be able to do in the course of a routine calculation. Various assumptions can surely be made, Turing argued, without imposing any essential restriction on what the clerk can do. First, he said, it's surely harmless to insist that the clerk writes on squared paper, and that within each square he may write just one of some *finite* collection of distinguishable symbols (like, for instance, those in our glyph alphabet). If an infinite repertoire of symbols were allowed, then some of these would be so similar to others that the clerk would no longer be able to distinguish them. Or, to take a more modern-day version of this idea, we can suppose the clerk is allowed to draw on a touch-sensitive computer screen in any way he likes (on a virtual pane which we suppose to be scrollable without limit in all directions). The shapes he draws will appear on the screen, which is subdivided into tiny squares called *pixels*, each of which can be (let's say) either black or white. If the pixels are small enough, the screen will easily be able to capture what the clerk has written down to the level of detail which he himself is able to see. Whichever of these models you prefer, we have the idea

that the clerk's 'paper' or 'workspace', though potentially unlimited in extent, can be supposed to be made up of squares which each contain only a finite piece of information.

Besides what's on the 'paper', there are two other ingredients that are relevant at any given point in time. One is the *position* on the paper that the clerk is currently looking at, as given by, say the square he has his finger on, or the portion of the computer pane that's currently in view. The other is the clerk's own *state of mind* at that point: this will include, for instance, his own mental note of where he's up to in the sequence of instructions he has been given, which symbol he was just about to write on the paper, where he was to write it, and perhaps a few other things as well. Turing argued that for our purposes, it's safe to suppose that only a *finite* number of distinguishable states of mind are possible: once again, if infinitely many states of mind were admitted, then surely some of these would be so 'close' to others that our poor clerk would become confused. Turing's idea was that it should never be necessary for the clerk to hold more than some small, bounded amount of information in his head: if anything more than this needed to be 'remembered', he could be told to write it on the paper instead.

We thus have the following picture: at each step of the computation, what the clerk does can depend only on three things: his current state of mind, his current working position on the paper, and the finitely many symbols within his current 'field of vision'. Depending on these things, he might (or might not) write or modify some symbol on the page, and might (or might not) then shift the focus of his attention to some other part of the page. His state of mind will probably also change, to reflect the fact he has now progressed to a different point in the computation. Turing argued that *whatever* particular computing instructions the clerk is following, what he does must surely have this basic character.

Now, this description of the clerk's activity is already quite mathematical in flavour, and by tightening it up just a little more, we arrive at almost exactly the definition of a *Turing machine*.[7] In other words, Turing arrived at his machines precisely by thinking hard about the ways in which an idealized computing clerk could actually work. For this reason, there's a conceptually compelling case that Turing machines can do everything clerks can do — which isn't immediately the case for (say) our tube machines, or for Church's lambda calculus. Indeed, for many people (including the logician Kurt Gödel, of whom more anon), it was Turing's arguments, rather than Church's, that clinched the case in favour of the thesis.

Turing's argument has since been tightened up a little further by other people, to the point where the thesis can actually be deduced as a theorem from some very basic assumptions about the nature of mechanical computation which seem easy to accept.[8] Indeed, the case for the Church-Turing thesis is nowadays felt to be well-nigh unassailable — in fact, I don't

know anyone who seriously thinks the thesis might actually be *false* in the form in we've discussed, though there may still be a few people who think the content of the thesis is not sufficiently clear to be considered 'true'.

Where there *is* still room for debate, however, is in some of the variations on the Church-Turing thesis that it's interesting to consider. For instance, what if we're allowed to replace our human clerk by a physical device of any kind we like? Or by a human doing something other than following routine instructions? Here things start to get a lot more controversial, and we shall return to some of these questions towards the end of this chapter.

'A fundamental discovery'

Let's just pause at this point to reflect on what the work of Church and Turing achieved. Routine methods of calculation, or algorithms, have played a central role in mathematics for centuries. Euclid himself introduced a remarkably efficient algorithm (still used today) for calculating the highest common factor of two Whole numbers. The word *algorithm* itself comes from the name of the ninth-century Persian mathematician Muḥammad ibn Mūsā al-Khwārizmī, whose text on arithmetic described all the basic methods for adding, subtracting, multiplying, dividing and square-rooting numbers in the 'Arabic' notation we use today. From then right down to Hilbert's great Decision Problem for mathematics as a whole, and on into the computer era, the quest for *ways of calculating things* has always been at the very heart of mathematics. So it's hardly surprising that the work of Church and Turing, which actually proposed a precise mathematical definition of the idea of a 'calculable operation', was hailed at the time as a profound and momentous breakthrough. The logician Emil Post greeted it as 'a fundamental discovery in the limitations of the mathematicizing power of Homo Sapiens',[9] later writing that it 'may play a role in the history of combinatorial mathematics second only to that of the formulation of the concept of natural number.' Post might well have been right about this, given that the breakthrough paved the way for the development of modern computers, as we will shortly see.

What's perhaps especially remarkable is that the enormous complexity and variety of algorithmic operations known to mathematics can in principle be simulated by such a tiny set of basic ingredients. Tube machines illustrate this point well. Here we're given just four footlingly simple basic building-blocks: adding 1, subtracting 1, comparing with 0, and a fourth one (the 'Y' component) that just merges two tubes into one. Nonetheless, it turns out that simply plugging these trivial components together gives us the power, in principle, to compute *any* function on Whole numbers that's computable by any mathematical algorithm ever devised, or by any program in any computer language ever invented.

I say 'in principle' here for good reason, because the price we pay for such extreme simplicity is that our tube machines would be impracticably 'inefficient' for most computing tasks. Have another look at the little machine for adding two numbers which Emily constructed. If we want to use this to add (say) 7419 to 12968, the disc will have to go around the loop 7419 times — obviously an agonizing waste of time compared with (say) the method for addition you learnt at school. And if we want to *multiply* the same two numbers, we will have to perform 7419 separate *additions*, each of which will itself require 12968 'steps'. Needless to say, anyone who builds real computers would laugh at this idea — after all, the above multiplication can be performed in just a single step by modern microprocessors. Of course, if we were interested in efficiency, we could easily extend our kit with a few other basic tube types: say, with components for adding and multiplying numbers. But that would rather defeat the point we're trying to make here, which is that though such extensions might well improve the *efficiency* of our tube machines, they don't *in principle* allow us to compute any additional operations that we couldn't compute with the basic kit.

In just the same way, there's nothing magical about the fact that our tube machines happen to have 9 registers — we could increase this to 10 or to 1,000,000,000, or decrease it to 5, without changing the class of functions we're able to compute.[10] Of course, for the purpose of actually building tube machines, the more registers we have, the easier life becomes; but for the purpose of studying them theoretically, there are advantages in keeping the number of registers quite small. My choice of 9 registers represents a balance between these two factors which is reasonably convenient for the investigation of of our next topic: the idea of a *universal* computing machine.

Turing's universal computer

I have just taken a quick look round my kitchen. Like most home kitchens in Britain, it's kitted out with various devices and appliances: a kettle, a toaster, a pair of old-fashioned weighing scales, a cooker, a washing machine, and a few others whose precise function I am rather hazy about. Obviously, the reason we have all these different devices, rather than just one, is that there are different jobs we want them to perform: the washing machine isn't particularly good at making toast, nor the cooker for measuring out margarine. But could one imagine a *single* kitchen appliance that fulfilled the role of all the devices I've mentioned, as well as those I haven't?

Of course, some of these devices *can* do duty for others — the cooker can play the role of either a kettle or a toaster — but arguably that's just because what I'm calling a 'cooker' is really various different devices assembled together. In the same way, you could glue a washing machine and a toaster

together and christen the resulting gadget a 'washing toaster' — a marvellous new gadget that *both* does the laundry *and* makes the toast. But that's not really the kind of thing I'm getting at, because each time you thought of a new task that you wanted your wonderful machine to perform (say, washing the dishes), the chances are you'd have to bolt on some extra bit of machinery for the purpose. Could there be a machine — some advanced kind of robot, perhaps — that was in some sense able to carry out *any* reasonable physical task, subject to some reasonable limitations? For all I know, such a thing might be possible in the future, but it's not on the horizon at the moment. As things stand today, we take it for granted that different machines are needed for different purposes.

But let's move now from my kitchen to my study — from the realm of physical tasks to that of information processing. Here, on my desk, we find a *single* device that is capable of the most astonishing range of tasks: playing videos, checking spellings, processing spreadsheets, transposing music scores, simulating planetary orbits, discovering prime numbers, playing chess, generating the index for a book I'm currently writing, and so on — not to mention the thousands more tasks it can perform in conjunction with other similar devices worldwide. What's more, if I think of something new that I'd like this device to do (such as removing background noise from an audio file), I can very often endow it with this new ability without any *physical* addition to the device at all, simply by feeding it with *pure information* — typically with just a long sequence of 0's and 1's sucked in from somewhere on the Internet. If this doesn't astonish us in some way, that's only because we've got so used to it. It's interesting here to note a statement made in 1956 by Howard Aiken, the designer of an 'Automatic Sequence Controlled Calculator' built by IBM for Harvard University:

> If it should turn out that the basic logics of a machine designed for the numerical solution of differential equations coincide with the logics of a machine intended to make bills for a department store, I would regard this as the most amazing coincidence I have ever encountered.[11]

What lies at the root of this amazing versatility of modern computers? Whilst it would be too simplistic to attribute this to just a single factor, there is nonetheless, at the heart of this phenomenon, a genuine mathematical insight: namely, the existence of a *universal* computing machine. This discovery, due to Turing and also included in his 1936 paper, takes us a step beyond the ideas we've already looked at in connection with the Church-Turing thesis. Let's look again at what this step was.

In the first part of our story, Anna and Emily found that any algorithm they came across could in practice be mimicked by a tube machine. But like a manufacturer of kitchen appliances, they found themselves having to build a

brand new tube machine for each task they wanted to perform: one for adding numbers, one for finding their prime factors, one for answering Real number glyph questions and so on. In just the same way, several people before Turing had built calculating machines for certain specific tasks. Around 1642 Blaise Pascal designed and built a mechanical device that could add and subtract — perhaps the world's first ever digital calculator. In 1671 Gottfried Leibniz topped this with a machine that could multiply and divide as well. And in 1822 Charles Babbage built a prototype of a 'Difference Engine' designed for producing accurate tables of logarithms and trigonometric functions. But for each new computing task, a new machine had to be built.

Actually, Babbage did take a fundamental step beyond all this. From 1834, he designed (but never built) his so-called 'Analytical Engine' — a fantastical configuration of Victorian machinery that would have filled a large hall, and which would have been 'programmable', by means of punched cards, to undertake a remarkable range of *different* calculating tasks. In this way, Babbage hit on the key idea that instead of building a separate piece of 'hardware' for each computing task, one could make do with a single piece of hardware and use it to run many different pieces of 'software'. Certainly, the potential power and versatility of the Analytical Engine were quite evident to Babbage — and even more so to his friend Ada Lovelace, who wrote with eloquence about the Engine and published some interesting 'programs' for it. But neither Babbage nor Lovelace clearly formulated the bold idea that the Engine could actually be programmed to perform *any* purely algorithmic computing task in principle.

It was Turing who recognized that a single machine — a *Universal Machine* as he called it — could suffice for all the mechanical computing tasks you could ever think of. It doesn't make too much difference here whether we think in terms of Turing machines or tube machines, as the essential ideas are the same in either case — so let's go with tube machines. In a nutshell, a Universal Tube Machine is simply a tube machine that can *emulate* any tube machine we please (even itself). We can think of such a Universal Machine as requiring two inputs. The first is some kind of formal description of the machine we're wanting to emulate (let's call this the *object machine*), while the second is simply the input we're wishing to supply to the object machine. Put slightly differently, we can think of the first input here as the 'program' or 'software' that we're running on the Universal Machine, and of the second input as the 'data' that we're running this program on.

Given these two inputs, the Universal Machine then calculates how the given object machine would behave when supplied with the given input, and then outputs whatever the object machine would output as a result. In this way, the Universal Machine, with appropriate programming, can emulate any object machine. And if we've already accepted the Church-Turing thesis, in the form that any mechanically computable function is computable by some

tube machine, then it follows that any such function can be computed by the Universal Machine.

The point about needing a *formal description* of the object machine deserves some comment. In order to tell our Universal Tube Machine which object tube machine to simulate, we'll need some convention by which any tube machine can be represented by just a single number. The precise details of how this is done don't really matter, as long as it's straightforward to translate back and forth between tube machines and the numbers that represent them. One fairly simple example of such a convention or 'coding scheme' — the one Turing used in our story to produce the long numerical code for Emily's adding machine — is described in a supplement, *A Universal Tube Machine*, on the book's website. This supplement also offers a series of exercises that take you through the construction of an actual Universal Tube Machine, if you are interested in doing this and have a few hours to spare.

In Turing's 1936 paper, the idea of such a Universal Machine was introduced just as an abstract mathematical possibility. But it wasn't long before people started thinking about how one might actually build such a machine in practice. Early in 1945, John von Neumann, then working at the Moore School of Electrical Engineering in Philadelphia, produced a landmark document outlining the design of a machine called the EDVAC (never actually built) which in essence embodied Turing's idea of a programmable Universal Machine.[12] Later that year, Turing himself produced a detailed design for a rather different kind of Universal Machine called the ACE, a pilot version of which was later built at Britain's National Physics Laboratory. Since that time, of course, the development of general-purpose programmable computers (and ways of programming them) has come on in leaps and bounds, to the point where many of us routinely carry around such machines in our pockets.

Of course, the kinds of computers that people actually build differ from Turing's idealized machines (or our tube machines) in important ways. On the one hand, any actual computer will in practice have only a finite amount of memory or storage space, whereas our 'theoretical' machines are considered as being able to store an unlimited amount of information. (We've seen that our tube machines can only 'remember' nine numbers, but the crucial thing is that these numbers can be as big as you like — they can run to octillions of digits — and in this respect our machines are more powerful than any physical computer ever built.) On the other hand, our theoretical models, which concentrate on computations with a single input and a single output, hardly do justice to the richly *interactive* nature of what real computers now do all the time, communicating as they do with human users, other computers and devices of many other kinds in fluid and complex ways. But even allowing for all of this, it's still fair to say that if you are to ask how the smartphone in your pocket is capable of such an astonishing range of tasks as it is, the heart of the

explanation lies in Turing's epoch-making paper which he submitted shortly before his 24th birthday.

On a more human note, I feel I can hardly go on without touching briefly on the other events of Turing's life, and on the tragedy of his early death.[13] During the war, Turing put his mathematical gifts at the service of his country by designing a machine that cracked the German 'Enigma' code, as movingly portrayed (albeit with much historical licence) in the 2014 film *The Imitation Game*. He is also remembered today for his revolutionary insights into developmental biology or 'morphogenesis', being among the first to suggest a possible mechanistic explanation for plant development. The story of his final years, however, is heart-rending. In 1952, he was convicted of having an affair with another man — at that time a criminal offence in Britain — and subjected to injections of oestrogen in an attempt to curb his sex drive. He died two years later, at the age of 41, of cyanide poisoning. The coroner's verdict was that he had committed suicide, although the veracity of this is still debated. Whatever the truth, it was a tragic and untimely end to the life of one of the 20th century's most deeply original thinkers.

In 2009, the British government publicly apologized for the appalling way Turing had been treated, and the Queen pronounced him officially pardoned in 2014.

Undecidable problems

As we've seen, the ideas of Church and Turing led in due course to spectacular practical applications. But they also led to remarkable advances on the purely mathematical side — in particular, to a definitive solution to Hilbert's question which we met in Chapter 2. Recall that Hilbert had asked whether there was a mechanical procedure that could be used to determine the truth of *any* mathematical statement.

It's important to stress that the name 'Universal Machine' doesn't imply that such a machine is capable of solving *every* mathematical problem that can be devised — only that it can solve every problem *that can be solved by mechanical means*. Indeed, one of the very ideas that went into the construction of the Universal Machine — the idea of representing an entire machine by a single number — can also be used to give a specific example of a problem that *can't* be solved by such means: namely, the problem of deciding whether a given computation ever comes to a halt.

As we saw in the story, we can prove that this problem — the so-called 'halting problem' — can't be solved by any tube machine, by first supposing that it can, and showing that this leads to a paradoxical conclusion. I won't repeat the argument in detail here, but the important point is that it gives us *one example* of a problem known to be mechanically unsolvable.[14] And once

we've found one such example, it becomes a whole lot easier to find others: we can show that some problem X is unsolvable by showing that if we *could* solve X, that would lead to a way of solving the halting problem as well.

Hilbert's decision problem gives a clear example of this. If there were a mechanical procedure that could solve *all* mathematical problems, then that procedure could certainly be applied to solve the halting problem, because the halting problem is a clear example of a mathematical problem in the sense Hilbert had in mind.

We can tighten up this argument a bit if we replace 'all mathematical problems' here by 'all Whole number glyph statements'. Because Whole number glyph language will play such an important part in the rest of the book, it's worth giving it a better name, so from now on I shall refer to it as the language of *Arithmetic* (with a capital A).

It turns out that with a bit of effort, statements about whether some tube machine computation halts can be mechanically translated into statements of Arithmetic. If you are now a fully qualified glyph wizard, you might like to work through the exercises in another supplement, *Talking about tube machines in glyph language*, to see how this is done; or you may prefer to take it on trust. In any case, once we know this, it's clear that if there were a tube machine that could answer all questions of Arithmetic, we could get it to solve the halting problem. But since this is impossible, we can conclude that there's no mechanical way to decide statements of Arithmetic, let alone any more general classes of mathematical problems. In other words, the class of true Arithmetical statements is an example of an *undecidable* mathematical theory. (You may find it helpful at this point to refer back to the Venn diagram at the end of Chapter 3, to see how things are fitting together.)

You might wonder at this point whether the whole issue of unsolvability is something that has crept in just because the problems we've been contemplating have become too ambitious. If we insist on asking such heaven-storming questions about 'all mathematical statements' or 'all mechanical computations', surely it's not surprising if we hit a roadblock sooner or later? Perhaps unsolvability is something that only arises at these 'outer limits' of mathematical thought — a metamathematical curiosity of little relevance to ordinary mathematics? Do the sorts of problems that mathematicians normally work on ever turn out to be unsolvable?

In the early days of computability theory, unsolvability might well have appeared as a somewhat esoteric phenomenon at the farther reaches of mathematics, of interest only to logicians. Since then, however, it has become clear that even surprisingly simple problems in ordinary mathematics can sometimes turn out to be mechanically unsolvable. It's worth looking at one particularly simple and important example, if only to emphasize how unsolvable problems can arise even from the most humble mathematical ingredients.

The problem in question is one mentioned by Hilbert in a famous list of 'challenge problems' he proposed in 1900. It has to do with whether certain rather simple kinds of equations have Whole number solutions. For example, take the following equation, which involves the variables x and y:

$$x \times x \ + \ y \times y \ = \ 13$$

This particular equation *can* be solved by Whole numbers: for instance, we can take $x = 3$ and $y = 2$, and the equation will be satisfied. But now let's tweak our equation just slightly:

$$x \times x \ + \ y \times y \ = \ 14$$

It's not too hard to see that this equation *can't* be solved by choosing Whole number values for x and y. (It can be solved if we allow Real numbers, but that's another matter.) So the question arises: given an equation like this, is there a way of telling whether it can be solved using Whole numbers or not?

Let's make the question a bit more precise. We're thinking about equations in which both sides are algebraic expressions that can be built up from Whole numbers and variables using just the operations + and × (as many times as we like). Here's a random example of such an equation involving five different variables:

$$x \times y \times z \ + \ 3 \times y \times y \ + \ 7 \times z \ = \ x \times z \times v \times w \ + \ 2 \times w \times w \ + \ 432 \times v$$

Equations of this sort, where we're looking for Whole number solutions, are known as *Diophantine equations*. The question is whether, given such an equation, we can tell, just by following some mechanical recipe, whether or not there's a solution using Whole number values for the variables.

Compared to the sorts of Real number problems that can be solved by Tarski's algorithm, this problem looks very simple indeed. And it is modesty itself alongside the decision problem for Arithmetic which Emily was so convinced ought to be solvable. All the more remarkable, then, that even this simple and natural problem should turn out to be mechanically unsolvable.[15] The proof of this fact essentially consists of a clever way of translating the halting problem into the Diophantine equations problem, implying that if the latter problem were solvable then the former would be too.

This is a good example of a simple unsolvable problem from classical mathematics. There are also natural, fundamental questions in areas such as group theory and topology that have been shown to be unsolvable, ultimately by relating them to the halting problem. Thus, the concept of unsolvability, whilst originating within mathematical logic, turns out to be of relevance to many other areas of mathematics.

A physical Church-Turing thesis?

The results we've talked about show that there are limits to what can be achieved by what we're calling 'mechanical calculation'. Since all calculations that present-day computers can perform are of this kind, these limits therefore apply non-controversially to what today's computers can do — and even to what they *could* do if they were given unlimited memory and computation time. But does this mean that we shall never be able to build a computer *of any sort* that exceeds these limits? After all, tomorrow's 'computers' might look quite different from today's in the physical processes they exploit: there might be quantum computers, biological computers and who knows what else. Does the Church-Turing limit apply to any possible physical device whatsoever? Or are there physical processes in nature which might conceivably be harnessed to go beyond the power of Turing machines?

What we're in effect asking here is whether we buy into the following 'physical' version of the Church-Turing thesis:

> *Any operation on Whole numbers that can be computed by any physical device whatsoever can also be computed by a Tube-Machine-in-a-box.*

This question has been discussed on and off since the early days of the subject,[16] but has received renewed interest in the last couple of decades. There are two kinds of reasons why one might be interested in the question. On the one hand, one might genuinely wonder whether some future technology could *in practice* give us a computer that 'broke the Turing barrier'. Probably only a tiny minority of physicists and computer scientists would hold out any hope of such a thing. But even if one doesn't, the question of the physical Church-Turing thesis surely has a certain philosophical interest. Do we live in a universe which, *in principle*, has what it takes to perform super-Turing computations? Or does our universe work according to physical laws whose very nature precludes this?

The case for the latter might at first seem very compelling. After all, an enormous wealth of scientific experience appears to support the view that the physical universe, insofar as it conforms to definite laws at all, conforms to laws of a computable, 'mechanistic' character. True, one can always speculate that there might be strange 'non-computable' things going on that we haven't discovered yet — or perhaps argue that there's a systematic reason why the laws we happen to have discovered so far are of such an eminently 'computable' character. However, we can only sensibly talk about what we do know, not what we don't; and as far as our existing physical theories are concerned, it might well appear, at a first glance, that they offer no scope for non-computable or super-Turing behaviour.

But is this really true? Does a detailed scrutiny of the theories in question support this conclusion? After all, if it really is the case that a certain physical theory is incapable of hosting super-Turing computation, then shouldn't it be possible to formulate and prove a mathematical theorem confirming this? On the other hand, if there's something in the theory that *does* admit super-Turing behaviour, then this too can be mathematically investigated. Such questions provide us with something definite to get our teeth into, bringing the discussion into the realm of genuine scientific inquiry.

And the problem is that when we try to do this, we quickly discover that the whole situation is far more complex and subtle than we might have suspected. For one thing, a host of questions arise as to what our 'ground rules' ought to be: in short, questions as to what should 'count' as a genuine super-Turing computation, and what kinds of 'idealizations' should be allowed.[17] But beyond this, it appears surprisingly hard to find *any* non-trivial and natural physical theories for which we can rigorously prove some form of 'physical Church-Turing thesis' without running into one bizarre problem or another.[18] Indeed, such investigations tend to throw up all sorts of quirks and curiosities accompanying existing theories, and even, in some cases, some fantastical thought experiments that might be seen as theoretical candidates for super-Turing computation.

This whole area is still very much in its infancy, and it seems to me that it offers exciting opportunities for further research. Perhaps the map will look quite different once the territory has been explored more fully. But in the meantime, I'd simply like to offer a few intriguing fragments, centring mostly on two of the thought experiments that have been proposed, and leave you to make of them what you will.

The first of these is definitely in the 'curiosity' category: it's not a serious contender for a physical super-Turing computer, as the underlying model is so easy to pick holes in, but it does illustrate the sort of bizarre scenario that can arise within even the tamest physical theories, and which can frustrate our efforts to establish a Church-Turing thesis even where one is expected. By contrast, our second proposal is perhaps the striking candidate to date for a theoretically possible super-Turing computer according to our current understanding of physics — albeit not one that anyone is likely to construct within the next 10,000 years.

Cats and catapults

Let's start with the world of *classical* physics — in particular, with Newton's theory of gravitation, which explains the way stars and planets move through space under the influence of gravity by means of a single, neat equation. This theory can be used to compute the orbits of the planets and

moons in our solar system with magnificent precision, and has long been regarded as the very paradigm of a scientific theory yielding orderly, predictable behaviour. So you might suppose that if we were wanting to formulate and prove physical versions of the Church-Turing thesis, then doing this for the 'clockwork universe' postulated by Newton's theory ought to be a fairly easy warm-up.

However, it turns out that even Newton's universe has its wild side, and can play host to some very unexpected behaviour. Of course, the universe we live in isn't a Newtonian one, and the scenario we'll be looking at certainly *won't* be compatible with what we know about our own universe. But that isn't really the point of the example. The point is just that if we're hoping to prove some physical Church-Turing thesis, we may have to be very careful about how we set things up, because it's much easier for non-Turing-computability to slip in by the back door than one might have supposed — even for something like our clockwork Newtonian theory.

In 1989, Joseph Gerver discovered an astonishing consequence of Newton's theory. It's possible to specify a symmetrical, circular configuration of 'stars' and 'planets', with delicately chosen initial positions and velocities, which, according to Newton's law, would perform the miraculous feat of disappearing off to infinity within one minute! Obviously, such a thing isn't compatible with the upper speed limit that's in force in our own universe (the speed of light) — but then a universe with such an upper speed limit wouldn't be a strictly 'Newtonian' universe. There are also other ways in which Gerver's scenario isn't physically realistic — in particular, the stars and planets have to be regarded as 'point masses' with zero diameter — but the point here is simply that what looks at first like an innocuous (albeit simplified) Newtonian model turns out to admit this surprising behaviour.

Later, in 2006, Warren Smith modified Gerver's scenario to yield a system which, in some sense, exhibits non-Turing-computable behaviour. Specifically, if the initial conditions of the system are just right, then the whole circle will expand to infinity as we've just described. But if we tweak these conditions just slightly and in the right way, then the system will expand for a certain distance before suddenly catapulting the planets inwards towards the centre at a terrifying velocity, arriving there before the end of (say) two minutes. In Smith's paper,[19] we are invited to consider the fate of an 'N-way symmetric, two-dimensional cat' who lives on an asteroid at the very centre of the circle. If the initial conditions are of the first kind, then the circle expands forever and the cat survives; but if they are of the second kind then the poor critter will be crushed to death by the volley of incoming planets. Either way, we'll know within two minutes what happens.

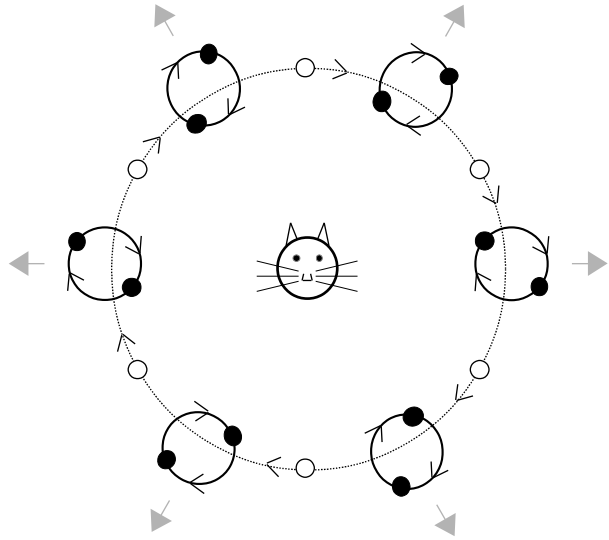


Figure 1: Scenario for the *Great Escape*: a symmetrical configuration of 'binary star' pairs (shown in black) and 'planets' (shown in white). If the initial setup is just right, Newton's theory predicts that the planets will circulate faster and faster, gaining energy from the binary stars whose orbits will get tighter and tighter. Furthermore, the entire circle will expand at an ever faster rate, shooting off to infinity within (say) one minute.

What's more, Smith shows that if we allow ourselves the ability to set up the initial conditions with arbitrary precision (in contravention of the second of my suggested ground rules), then we can actually use this to solve the halting problem. Specifically, we can map any instance of the halting problem (say, a particular tube machine plus input) to some set of initial conditions for our gravitational system, in such a way that if the computation continues forever, we are in the first situation and the cat lives; but if the computation ever halts, we are in the second situation and the cat suffers accordingly.

In spite of the long list of objections (both physical and ethical) that one might raise to this experiment,[20] it clearly has mathematical curiosity value, and it illustrates, in principle, how non-Turing-computability can unexpectedly insinuate itself into a physical theory. Of course, in this case, it's easy to suggest refinements to our model which would rule out the 'unrealistic' non-computable behaviour. On the other hand, each time we complicate our model, it would seem that we run a risk of admitting non-computable behaviour in some other way. Whether there is a good way out of this remains to be seen.

So much for Newtonian dynamics. Let us now turn our attention to the two great pillars of 20th century physics: quantum mechanics and relativity theory.

Quantum computers

You might have heard of an exciting new field called quantum computing. The idea here — which was propounded by David Deutsch in 1985 — is to make use of some of the strange effects described by quantum theory (such as the possibility of a particle being in a 'mixture' of two different positions simultaneously) to perform computations in exciting new ways that wouldn't be imaginable on a classical computer. Then in 1994, the mathematician Peter Shor stunned the scientific community by showing how a quantum computer would in theory be able to factorize large numbers far more efficiently than any known classical method (thus putting the security of the RSA cryptosystem in jeopardy). Since then, many other interesting and remarkable 'quantum algorithms' have been discovered, often improving dramatically on their classical counterparts, and the whole subject has become a vibrant area of research.

Are quantum computers realizable in practice? Over the last decade, huge investments have been poured into developing the technology required to make the theory a reality. The quantum computers so far constructed have all been quite small — as I'm writing this, the largest has a capacity of 72 'quantum bits', or *qubits* — but they're getting bigger rapidly. And even smallish quantum computers are interesting: once they exceed about 50 qubits, the task of simulating their behaviour on even the fastest classical supercomputers today becomes impractical. So they're already doing *something* we can't do by classical means, and the race is now on to harness this potential: to apply quantum computers to some well-defined computational task that would be out of the reach of today's classical computers.

In the light of what we've said so far, you might think that this would be exactly the place to look for a machine that 'broke the Turing barrier'. Sadly, this turns out not to be the case: although quantum computers promise spectacular gains in terms of the *speed* with which certain computations can be carried out, they don't provide the means to compute any totally new operations that couldn't *in principle* be computed on a clunky old Turing machine. This is because the particular bit of physics that's exploited by quantum computers (the Schrödinger equation) is actually just as 'computable' in character as anything in classical physics. So once again, if it's a super-Turing computer we're after, we shall have to look elsewhere.

Black hole computers

The picture changes when we turn to the other great breakthrough in 20th century physics. Einstein's theory of *general relativity* presents us with a

strange universe in which time may flow at different ‘speeds’ according the strength of the gravitational field you’re in: while one hour has passed on the earth’s surface, the time experienced by someone on the moon will be ever so slightly more. People have wondered whether this phenomenon, taken to its extreme, could somehow be used to buy ourselves an *infinite* amount of time, which could then allow for some kind of super-Turing computation. A particularly remarkable proposal along these lines has been put forward by the Hungarian mathematician István Németi and his colleagues. I’d like to describe this proposal in some detail, since it seems to me the most interesting and thoroughly developed candidate for any kind of super-Turing machine to date, and the one that perhaps comes closest to meeting all the requirements I suggested earlier.

The sheer audacity of Németi’s proposal is breathtaking. Let us say you want to solve some instance of the halting problem: that is, you’re interested in whether some particular tube machine computation ever finishes. (Actually, you are only going to get one shot at this, so you’d better choose some computation you’re really*, really* interested in — one from which you’ll learn something mathematically significant.) Here is what you have to do. First, you must find yourself a very large and slowly rotating *black hole* somewhere in the universe. Black holes are mysterious regions of space in which the gravitational field is so strong that nothing — not even light — can escape from them. You ‘park’ your spaceship in orbit around this black hole at a safe distance, and program a large conventional computer on board the spaceship to simulate the tube machine computation you’re interested in. If the computation in question ever halts, then eventually your computer will find this out. If the computation *never* finishes, then the simulation will simply continue forever.

But now here’s the clever bit. You yourself don’t remain on the spaceship: instead, you let yourself fall freely into the black hole! Your computer, meanwhile, has been programmed to send some recognizable light signal after you if its computation ever halts. By choosing your trajectory carefully, you can arrange that by the time your wristwatch shows (say) 5 p.m., it will appear to you as though an *infinite* amount of time has elapsed on your spaceship outside. Which means that if you haven’t received the agreed signal by teatime, you’ll know that the computation never finished! Either way, from your perspective, you’ll know the answer within a finite time.

Actually, there’s a bit more to it than this, and to understand the idea, we’ll need to know a bit more about black holes. At some point, you will cross the black hole’s *event horizon* — this is the point beyond which it will be physically impossible to escape from the black hole, or even to send a message back to the outside world. So Németi is effectively offering you a one-way ticket: crossing the event horizon is the price to pay for the infinite time dilation that the scheme depends on. In fact, if it were possible to

suspend yourself precisely on the event horizon for even a fraction of a second, that would already be enough to achieve the time dilation we want. Unfortunately, this *isn't* possible, because the gravitational force here would be too strong for anything to survive. But as long as you're in free fall, you won't suffer from this extreme force. True, you might feel somewhat stretched by a *tidal* force resulting from the difference between the strength of gravity at your head and at your feet; but if the black hole you've picked is big enough, this shouldn't be too harmful and might even be quite pleasurable. In fact, you won't see or feel anything unusual at all as you cross the event horizon. But then again, if you're *just* falling, you won't get the infinite time dilation effect. What we want is some further property of the black hole itself which allows us to benefit from this effect *and* survive.

This is where it becomes important that you've chosen a *rotating* black hole. Within such a beast, there will be two forces at work: the gravitational pull sucking you inwards, and something like a *centrifugal* force pushing you outwards, in the way that clothes are pushed against the sides of a spin drier. In fact, the outward force will gradually increase in strength as you continue your journey inwards, until you reach a second 'event horizon' where this force exactly balances the inward pull of gravity. By this critical point, you will indeed have received any light signal sent by the spaceship — even if it was transmitted after 10^{1000} years of computer time according to the spaceship's clocks.

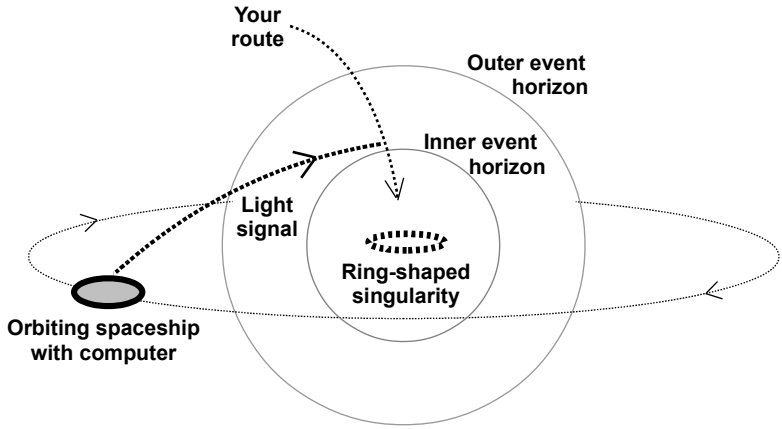


Figure 2: Németi's proposal for breaking the 'physical Church-Turing thesis' with the help of a large, slowly rotating black hole. However long the computer takes to complete its computation, its light signal will reach you before you reach the inner event horizon.

There's a minor problem with this whole scheme which you might have spotted. Not only are you now on the inside of a black hole with no hope of returning home, but you are still hurtling towards its centre at a terrifying velocity. Surely this means you'll have only a few seconds to enjoy the benefit of your newfound mathematical knowledge before your very atoms are ripped apart by the mysterious singularity at its core?

Not necessarily, says Németi. You've done your astrophysics homework, and have selected a kind of rotating black hole in which the singularity is not so much a single 'point' as a kind of 'hoop'. By arranging that you fall into the middle of the hoop in the right way, it becomes possible for you to avoid the singularity indefinitely. As Németi says, you can take your friends with you, you can found a society, and you can continue to develop your mathematical and scientific knowledge on the basis of what you've learned from your 'infinite computation'. If that's what you're wanting from life, of course.

There are various other problems raised by this proposal, but Németi and his co-workers have sought to address all of these and have suggested some ingenious measures to get round the various difficulties.[21] I'm not enough of a physicist to be able to judge these proposals for myself, but the work is impressively thorough and supported by detailed calculations. It would seem from this work, then, that a super-Turing computation of this sort is, at the very least, *not incompatible* with what we currently know about the universe we're in.

One final twist to this story. Although these 'black hole computers', if they existed, would be able to solve problems beyond the reach of tube machines, there would still be further problems that even *they* couldn't solve. For instance, a black hole computer could solve the halting problem for ordinary tube machines — but what about the halting problem for black hole computers? It turns out that the argument for the undecidability of the halting problem is a very general one, and can be applied to *any* reasonable class of computing devices, real or hypothetical: for any such class C, there can be no C-machine that solves the halting problem for C-machines. So in a certain sense, the mere idea of a machine that could solve 'all' problems turns out to be a logically incoherent one.

Uncharted waters

So much for what's possible according to general relativity. Are there any other areas of modern physics that hold out the possibility of breaking the Church-Turing limit? At this point, things get highly speculative, because we enter territory in which the physics itself isn't fully worked out yet. A big question in present-day physics is how quantum mechanics and general

relativity fit together — there are lots of ideas around, but no consensus among the experts as to which (if any) are headed in the right direction. But at least some of these ideas have involved mathematics that goes beyond what is Turing-computable: for instance, a proposal for a theory of *quantum gravity* by Robert Geroch and James Hartle involves a question about four-dimensional spaces that's known to be Turing undecidable. As I say, all of this is highly speculative — the point is simply that there's a lot we don't know, and there would seem to be no strong arguments, at present, either for or against a physical Church-Turing thesis for the ultimate 'Theory of Everything', whatever that might turn out to be.[22]

Of course, even if there *is* something non-Turing-computable going on in these uncharted regions of physics, it doesn't automatically follow that there's a way to harness it in order to achieve something that can't be achieved by Turing computation. Nevertheless, one astonishingly bold suggestion has been put forward as to where this might actually be happening.

In the late 1980s, one the most outstanding scientists of our time, the mathematician and theoretical physicist Roger Penrose — already well known for his far-reaching work on black holes amongst other things — advanced the daring hypothesis that some kind of non-computability in the physical world is harnessed by something very much closer to home: our own brains! What's more, Penrose suggested that something of this kind might play an essential role in our capacity for conscious understanding. These ideas were developed in a best-selling book, *The Emperor's New Mind,* and a later, more closely argued book, *Shadows of the Mind*.

What was the reasoning that led Penrose to this brave conclusion? Remarkably enough, his leading argument came not from physics or from neurobiology, but from mathematical logic. The argument was a variant of one previously advanced by the philosopher John Lucas, and purported to show that our human faculty for recognizing mathematical statements as 'true with mathematical certainty' could never be fully replicated by a Turing machine. This led Penrose to ask where such non-Turing-computable behaviour in the brain might come from, and to suggest that some as yet unknown theory of quantum gravity might provide the answer.

Penrose's books provoked a storm of controversy, and his arguments were severely criticised on a number of fronts by logicians, philosophers, physicists and biologists. Penrose offered numerous rejoinders to his critics, but the majority of scientists remained unconvinced, and the ideas did not gain a wide following. The debate over these ideas is still going on, although much of the attention has now shifted to some specific hypotheses, developed by Penrose in collaboration with the neurobiologist Stuart Hameroff, concerning exactly where in the brain this strange quantum or 'post-quantum' behaviour might be taking place.[23]

I've decided to devote a significant amount of space in this book to the Lucas-Penrose arguments, and to the various criticisms levelled against them (and to the criticisms of the criticisms, and so on). This is not because I myself am totally convinced by these arguments, but because I think they provide an excellent focus for discussion, bearing as they do on many of the fundamental questions at the very core of this book. Indeed, we shall find that our efforts to get to grips with what's really going on here will carry us quite deep into the heart of mathematics itself. We'll look at the Lucas-Penrose arguments in detail in Chapter 8, returning to them again in Chapter 11. But before we get there, we'll need to become familiar with a few other key ideas. In the next chapter, we'll look at how we can give a precise treatment of another of the basic ingredients of mathematics: the laws of correct logical reasoning.

Endnotes

1. The two historic papers are: A. Church, *An unsolvable problem of elementary number theory*, American Journal of Mathematics 58 (1936); and A.M. Turing, *On computable numbers, with an application to the Entscheidungsproblem*, Proc. London Mathematical Society 42 (1936). Both papers are reproduced, along with many other important papers from the early days of computability, in Davis (1965), an essential resource for serious students of the subject.
2. This view of Church's thesis — that it is a philosophical claim rather than a purely mathematical one, and as such is not amenable conclusive proof — is the traditional one, and has support in the writings of Church and Turing themselves. Church referred to the 'somewhat vague intuitive notion' of effective calculability; and Turing stated that all possible arguments for the thesis 'are bound to be, fundamentally, appeals to intuition, and for this reason rather unsatisfactory mathematically.'

   More recently, however, this view has been challenged by some: e.g. by Peter Smith (2007), who has argued that even the intuitive concept is actually sufficiently clear-cut to allow some kind of proof of the thesis. Even on Smith's proposal, however, one can (I think) distinguish between a 'pre-mathematical' phase of the argument in which the informal concept is analysed, and a purely 'mathematical' one which builds on key properties established by the first phase.
3. Geek alert: I'm following Church here in stating the thesis only for *total* numbers on the Whole numbers — those that yield a definite output value for every input number. However, everything I say here would apply equally well to *partial* functions, which would be the more natural setting for the thesis from today's standpoint.
4. The stronger claim — that a tube machine can be found which in some sense carries out 'the same algorithm' as the clerk — has been dubbed *Church's superthesis* by Kreisel (see Odifreddi's article Kreisel's Church in Odifreddi (1996)). Finding the best way to make this precise is a significant challenge, as we still don't have a good mathematical definition of 'algorithm' at an appropriate level of abstraction. But even without this, one can frame coarse-

grained versions of the superthesis to the effect that, for example, that the number of 'computation steps' carried out by the clerk and by the tube machine (or any other reasonable computation model) don't differ too wildly from one another.

5. See M.L. Minsky, *Computation: finite and infinite machines*, Prentice-Hall (1967). Similar ideas were also developed around the same time by Melzak and by Shepherdson and Sturgis. Minsky himself, incidentally, was a major pioneer in the field of artificial intelligence, and served as AI consultant for the character of the computer HAL in the 1968 film *2001: A Space Odyssey*.
6. For an early appeal to the 'convergence' argument, see J.B. Rosser, *An informal exposition of proofs of Gödel's theorem and Church's theorem*, Journal of Symbolic Logic 4 (1939). The challenge to this argument came largely from Georg Kreisel: see Odifreddi (1996), in particular the article on *Kreisel's Church*, which collects relevant references to Kreisel's work.
7. For mathematical convenience, a Turing machine is actually defined to be a device that works on just a one-dimensional strip of squared paper (in other words, a *tape*), and can only 'see' one square at a time. As Turing observed, neither of these simplifications make an essential difference to the power of the resulting machines. For an elegant description of Turing machines at a popular level, I recommend Davis (2000).
8. See for instance the discussion in Smith (2007). Also R. Gandy, *Church's thesis and principles for mechanisms*, in *The Kleene Symposium*, North-Holland (1980); and W. Sieg, *Church without dogma – axioms for computability*, in S.B. Cooper, B. Löwe, and A. Sorbi, editors, *New Computational Paradigms*, Springer-Verlag (2008).
9. See Footnote 8 of E.L. Post, *Finite combinatory processes. Formulation I*, Journal of Symbolic 1 (1936), reproduced in Davis (1965).
10. Perhaps surprisingly, if one is only interested in computing functions of a single argument (like the Fibonacci function mentioned earlier), one can get away with as few as 3 registers and still obtain all Church-Turing computable functions. (This is explained as part of the supplement *A Universal Tube Machine*.) For other purposes, however, 5 registers seems a natural minimum: for instance, a 5-register tube machine can achieve a component-by-component simulation of an N-register one (for any N), and 5 registers also suffice for the construction of a two-argument Universal Tube Machine of the kind described in this chapter.
11. This quotation appears in Davis (2000). As Davis notes, by 1956 Aiken really should have known better, since the ideas of Turing from 20 years earlier had by then already found their way into commercially available computers that could be programmed to do both these things.
12. The precise extent of the influence of Turing's ideas on von Neumann remains a bit unclear. See Davis (2000) and Hodges (2014) for discussion.
13. The magnificent biography of Turing by Andrew Hodges (2014) is highly recommended, both for its rich portrayal of Turing's life and for its detailed technical description of the Enigma code and Turing's method of cracking it.
14. As an essential 'must-read' for all serious students of this book, I warmly recommend Geoffrey Pullum's poem *Scooping the Loop Snooper*, a proof of the

undecidability of the halting problem written in the style of the American children's author Dr. Seuss. An Internet search will find it.

15. This was proved in 1970 by Yuri Matiyasevich, who supplied the final piece of a jigsaw built up earlier by Julia Robinson, Martin Davis and Hilary Putnam. See Y.V. Matiyasevich, *Hilbert's tenth problem*, MIT Press (1993).
16. See e.g. L. Kalmár, *An argument against the plausibility of Church's Thesis*, in A. Heyting, ed., 'Constructivity in Mathematics', North-Holland (1959); or G. Kreisel, *A notion of mechanistic theory*, Synthese 29 (1974).
17. The question of 'ground rules' seems to me important, as there have been a number of proposals for 'super-Turing computation' which (as it seems to me) really don't count, and which would be ruled out by some quite simple requirements. (See for instance H. Siegelmann and E. Sontag, *Computational power of neural networks*, J. Comput. Syst. Sci. 50 (1995), and the rebuttal in M. Davis, *The myth of hypercomputation*, in C. Teuscher, ed., 'Alan Turing: Life and Legacy of a Great Thinker', Springer (2004).)

    I would personally suggest some ground rules along the following lines. First, a super-Turing computer ought to be able to answer some specified mathematical question known to be beyond the reach of Turing machines — a device that merely tuned into the random background noise of the universe (if there is such a thing) wouldn't really be 'computing' anything. Second, we'd like a machine that could accept inputs and deliver outputs in some easily realizable form as in an ordinary computer: we shouldn't have to set up or measure anything with unattainable levels of precision in order to interface with the computation. Thirdly, for fairness, I'd suggest that we *should* allow the sorts of 'idealizations towards infinity' that we'd standardly allow in considering Turing computability (e.g. unlimited memory, time and patience). Of course, we might end up arguing over *which* idealizations were morally of this kind; but if we at least accept the spirit of the idea, we may be able to make progress.

    On a related point, it has sometimes been questioned whether, even in principle, any possible experiment could ever certify that a device was truly going beyond the Turing computable. (See M. Davis, *Why there is no such discipline as hypercomputation*, Applied Mathematics and Computation 178 (2006).) It seems to me, however, that we *can* conceive scenarios in which we would be led to believe that super-Turing computation was happening.

    As a flight of pure fantasy, for example, suppose someone built a machine that *seemed* to be a Decider for Whole number glyph statements, in the sense that over 100,000 trials on statements of up to 10,000 symbols, it always delivered a yes-no answer within 1 second, and these answers appeared totally consistent with its being a Whole number Decider as far as anyone could tell. Such a machine would, I think, seem so 'magical' to us that we'd have little hesitation in ascribing 'super-Turing' capabilities to it.

    It's mouthwatering to contemplate what we could do with such a machine. Not only could we obtain yes-no answers to open questions like the Riemann hypothesis (which we wouldn't know whether to trust), but by asking the right sequences of questions, one could (almost certainly) discover exciting new mathematical proofs that could then be verified by conventional means. So such a machine would authenticate itself as an 'oracle' delivering new knowledge not

previously available. I suspect that our most natural scientific hypothesis would then be that some kind of super-Turing process was indeed going on; moreover, extended experience with the machine would engender the same kind of trust in this hypothesis as we place in other physical theories.

Of course, this is all hypothetical in the extreme: I, for one, certainly *don't* expect that any such machine will ever be built!

18. As a matter of fact, I'm not currently aware of *any* natural examples of 'physical theories' that are rich enough to support classical Turing computation, but which have been proved not to admit anything beyond this. This may be just because not many people have tried yet. But it's interesting to note that a closely related question — whether familiar physical theories can be proved to be *deterministic* — has been probed in some depth, and all sorts of curious problems come out of the woodwork. See J. Earman, *Determinism: what we have learned and what we still don't know*, in J.K. Campbell (ed.), *Freedom and Determinism*, Cambridge, MA (2004), and the same author's earlier *A primer on determinism*, Springer (1986).
19. See W.D. Smith, *Church's thesis meets the N-body problem*, Applied Mathematics and Computation 178 (2006).
20. I've already mentioned the points about the speed of light and the assumption of point masses. Another serious issue is that an unattainable level of precision is required in setting up the initial conditions, *contra* the second of the ground rules that I've suggested. But my favourite quibble by far is that since Newton's laws are time-symmetric, there's nothing in the theoretical setup to exclude the possibility of a second 'circle' of stars and planets mysteriously appearing from nowhere (i.e. from infinity) and spiralling rapidly inwards, spoiling the experiment and killing the poor cat anyway.
21. See G. Etesi and I. Németi, *Non-Turing computations via Malament-Hogarth space-times*, International Journal of Theoretical Physics 41(2) (2002); and I. Németi and Gy. Dávid, *Relativistic computers and the Turing barrier*, Journal of Applied Mathematics and Computation 178 (2006). One group of problems concerns whether the computation (as seen from the outside) can really continue indefinitely. Specifically, *(a)* as the computation proceeds, the amount of memory the computer requires will tend to infinity; *(b)* the black hole may eventually 'evaporate' due to Hawking radiation; *(c)* the protons that make up the computer may decay after 10^{35} years. The proposed solution is to send scavenger robots out into the rest of the universe to search for additional matter/energy and beam it back to the home ship — this can be used to extend the computer's memory, keep the black hole stoked up, and occasionally rebuild the entire ship with fresh protons. It's claimed that according to at least some currently plausible cosmological models, all of this is OK.

    A different kind of problem is that when you receive the light signal it may appear so drastically *blueshifted* — that is, compressed into such a short timespan — that you are unable to detect it. The proposed solution is to compensate for this by redshifting the signal the computer sends — perhaps by transmitting it from another spaceship receding very quickly into a second black hole located conveniently nearby.

More recently, Németi and his colleagues have developed an alternative proposal for taking advantage of infinite timelines by making use of *wormholes* in space-time. This sidesteps the blueshift problem completely, but a major problem is that no one knows whether wormholes actually exist in nature or how to make one, although their existence is quite consistent with general relativity.

22. The trouble here is that at present, just about anything seems imaginable. One can certainly imagine a 'final theory' for which the physical Church-Turing thesis was a provable theorem (such a theory would of course have to rule out Németi's proposal in some way). But from a mathematical point of view, there's nothing implausible about a physical theory offering super-Turing capabilities, but that gave rise to a number of 'Turing computable' theories as special or limiting cases, accounting for why so many of our current 'approximating theories' have this character. This would be analogous to what we've already seen for the theory of Whole number glyph statements, which is Turing undecidable but has several important decidable sub-theories (see the end of Chapter 3).

For details of the quantum gravity proposal, and other remarks pertinent to our discussion here, see R. Geroch and J.B. Hartle, *Computability and physical theories*, Foundations of Physics 16(6) (1986). For a recent and intriguing sample of (widely divergent) perspectives on the relationship between physics and computation, see Zenil (2013).

23. Specifically, Hameroff and Penrose have suggested that structures known as *microtubules* within the brain's neurons may be capable of supporting the required behaviour. It's far beyond my scientific competence to judge the merits or otherwise of these proposals, but it's fascinating to watch how the debates are unfolding. For more information on this side of the story, see Hameroff's website: www.quantumconsciousness.org